



## ALGORITMOS E ESTRUTURAS DE DADOS - 2026/2027

### AULA DE LABORATÓRIO #03 - 5 A 9 DE OUTUBRO

#### MANIPULAÇÃO DE TABELAS E LISTAS LIGADAS

## Objetivos

Este laboratório estrutura-se em duas partes, cada uma com um objetivo específico. Na primeira parte o objetivo é discutir e comparar, mediante métricas simples de complexidade, soluções para um problema muito simples e desenvolver uma solução computacional para o mesmo. Não é fornecido qualquer código que se espera ser desenvolvido na aula, coletivamente.

Na segunda parte, para um problema igualmente simples, vão ser usadas e comparadas soluções baseadas em estruturas de dados diferentes, nomeadamente Tabela e Lista Ligada, e verificada na prática a gestão de memória alocada dinamicamente. Em ambos os casos é fornecido código base, **a ser completado**, para realizar as duas implementações, bem como um conjunto de dados de teste. Sugere-se comprovar as diferentes utilizações de memória nas várias implementações, recolhendo dados produzidos pelo Valgrind, e.g. `total heap usage: [...] NNN,NNN bytes allocated`

## 1 Primeira Parte

Considere o seguinte problema: é-lhe dada uma tabela,  $V$ , com dimensão  $N$ , de inteiros distintos e tal que para um dado índice  $p$  ( $0 \leq p \leq N - 1$ ) os valores da tabela crescem até à posição  $p$  e depois decrescem até à posição  $N - 1$ .

Pretende-se desenvolver uma solução eficiente para, dado  $V$ , determinar o valor do seu máximo,  $V[p]$  (note que o máximo pode estar num dos extremos da tabela). Para resolver este problema foi proposta a seguinte solução:

```
int findMaxUniM (int *V, int N) {
    int i, max = V[0];

    for(i = 1; i < N; i++) {
        if (max > V[i])
            break;
        max = V[i];
    }
    return max;
}
```

- Indique, de forma justificada, como função de  $N$ , a complexidade computacional da solução proposta. Para ser considerada completa, a sua resposta deve ser devidamente fundamentada e justificada.
- Desenvolva em C uma solução mais eficiente para o mesmo problema.

**Nota:** a sua solução tem de ter uma complexidade assintótica melhor que a apresentada!

**Sugestão:** Pense neste exercício como um problema de procura de um elemento numa tabela e nas soluções eficientes que estudou para esse caso.

- Como função de  $N$ , determine, justificando, a complexidade computacional da solução apresentada em b).

## 2 Segunda Parte

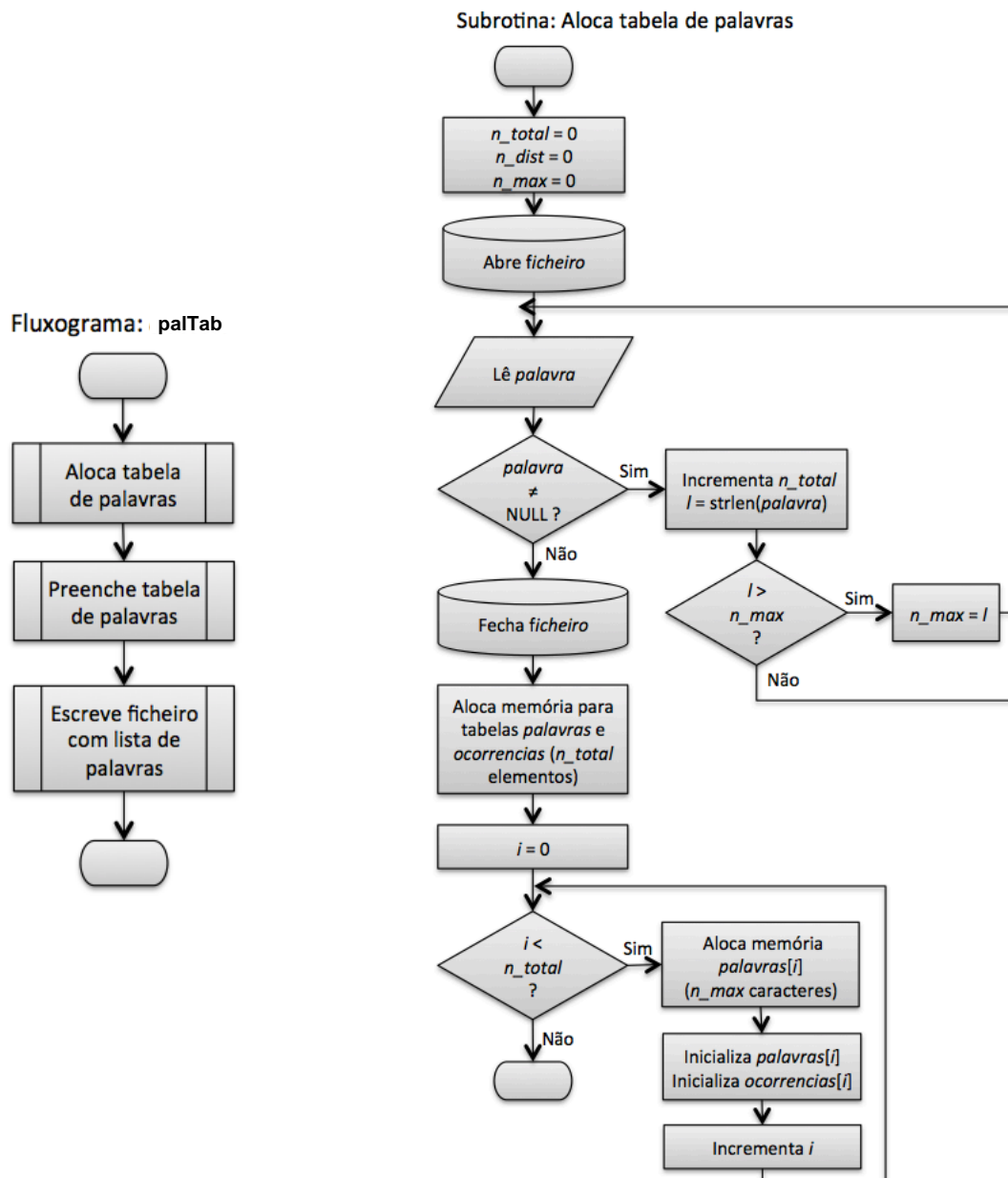
Considere um programa com a seguinte especificação:

UNIQPAL é um problema computacional em que se pretende ler de um ficheiro de texto e escrever num ficheiro resultado uma lista das palavras distintas encontradas, bem como o número de ocorrências de cada palavra. Cada palavra deve aparecer uma única vez no ficheiro de saída. O ficheiro de entrada é especificado na linha de comando; o nome do ficheiro de saída deve ser igual ao de entrada mas acrescentando `<.palavras>` no final (por exemplo, se o ficheiro de entrada for `xpto.txt`, o ficheiro de saída será `xpto.txt.palavras`).

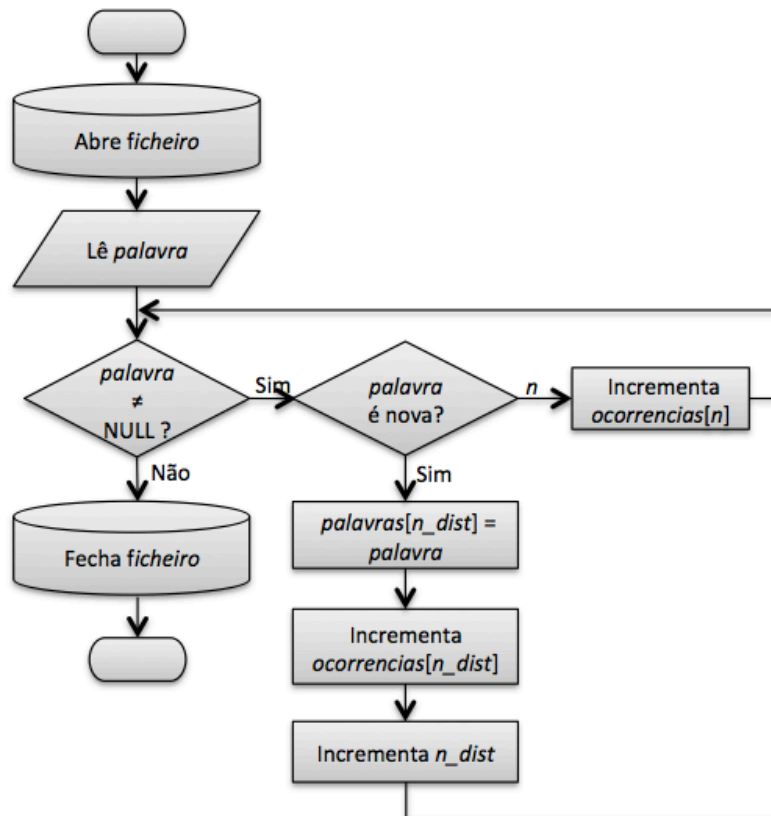
### 2.1 Manipulação de Tabelas

O ficheiro `palTab.c`, que se encontra na sua área de trabalho, contém uma solução para o problema descrito acima.

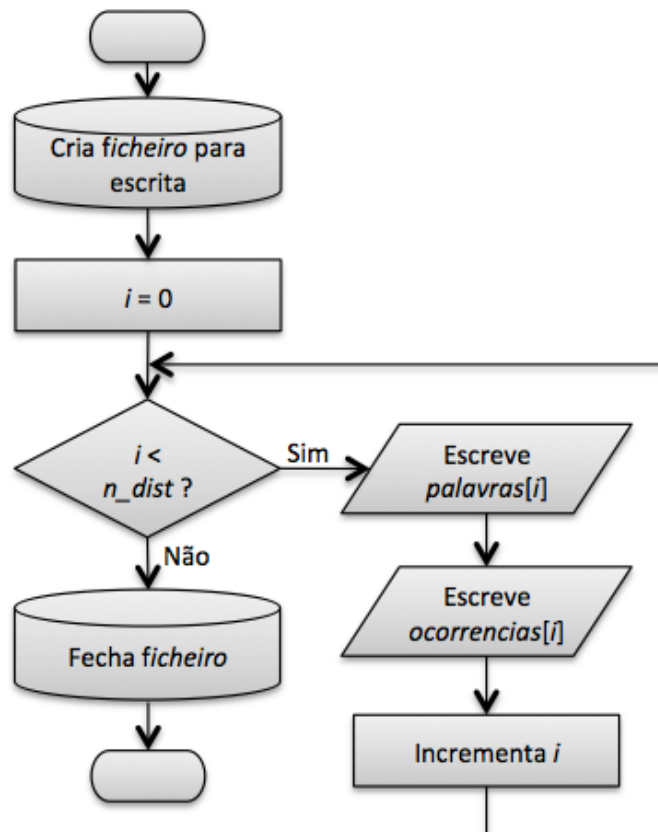
- a) Examine o código e o fluxograma correspondente (figuras abaixo). Repare na forma como o código se encontra estruturado em várias funções e que as palavras distintas lidas do ficheiro de entrada são guardadas numa tabela.



Subrotina: Preenche tabela de palavras



Subrotina: Escreve ficheiro



- b) Complete o programa fornecido com o código necessário para a alocação e libertação da memória dinâmica necessária para a sua execução. Corra o Valgrind para verificar que a gestão de memória dinâmica está bem efetuada.

**Exemplo:** `valgrind --leak-check=full ./palTab ../test/doc1.txt`

- c) O programa proposto, no ficheiro `palTab.c`, para concretizar o problema UNIQPAL faz uma gestão muito ineficiente de memória. Identifique as razões para essa gestão ineficiente e proponha uma solução que reduza a memória utilizada. Modifique o fluxograma fornecido em conformidade.

**Nota:** Não se pretende aqui alterar por completo a solução apresentada, nomeadamente pretende-se manter uma estrutura baseada em tabelas. Quer-se apenas reduzir o desperdício desnecessário da memória alocada. Na segunda parte do laboratório temos uma solução que poderá gastar menos memória com uma aproximação diferente ao problema.

- d) Altere o ficheiro `palTab.c` de acordo com a solução que propôs. Corra o Valgrind para verificar que a gestão de memória dinâmica está bem efetuada

## 2.2 Manipulação de Listas Ligadas

A versão desenvolvida para a primeira parte do laboratório baseia-se na utilização exclusivamente de tabelas com as vantagens e limitações daí resultantes. Uma alternativa consiste em guardar as palavras, não numa tabela, mas numa lista ligada, como veremos nesta parte do laboratório. Aproveitaremos esta oportunidade para desenvolver e compreender como utilizar abstração para separar a interface da utilização dos tipos de dados da sua implementação específica. Como exemplos deste processo apresentamos as implementações de Palavras (*words*) e “Lista Ligada”. Considere então o seguinte algoritmo:

```
Abrir Ficheiro_de_Entrada
Ler Ficheiro_de_Entrada
  Enquanto houver palavras:
    Ler palavra
    Testar se palavra é nova na lista ligada
      Sim: guardar na lista ligada e inicializar o número de ocorrências a 1
      Não: incrementar o número de ocorrências da palavra
  Fechar Ficheiro_de_Entrada
Abrir Ficheiro_de_Saída
Para cada palavra da lista ligada
  Escrever no Ficheiro_de_Saída o número de ocorrências, a frequência e a palavra
Fechar Ficheiro_de_Saída
Escrever no écran o número de palavras lidas e o número de palavras distintas
```

Nos ficheiros `main.c`, `words.c` e `list.c` é fornecido código, incompleto, que implementa o algoritmo indicado. Note as especificações adicionais nos pontos 2 e 3.

- a) Examine o código da implementação de lista ligada disponível em `list.c`. Veja como a implementação da lista ligada descreve uma abstração e está totalmente contida neste módulo, sendo apenas disponibilizadas, em `list.h` funções que oferecem funcionalidades básicas para este tipo de estrutura sem descrever a forma como as mesmas são implementadas.
- b) Examine agora o código de `words.c` que contém uma implementação de uma estrutura de dados que permite guardar as palavras de acordo com o pretendido no problema (para cada palavra guardar apenas uma cópia da mesma e mantendo a informação do número de ocorrências de cada uma no texto lido). Mais uma vez a implementação está totalmente contida no módulo sendo apenas disponibilizadas, em `words.h`, funções de criação, destruição, adição de uma palavra às existentes, impressão de todas as palavras armazenadas e obtenção do número de palavras armazenadas (obviamente outras funcionalidades podiam ser codificadas). Uma aplicação que utilize este módulo desconhece por completo a implementação do mesmo e tem apenas acesso às funcionalidades mencionadas.

- c) Faça um fluxograma do programa. Por uma questão de legibilidade sugere-se que, tal como foi feito no Problema 2.1, se considere a possibilidade de fazer fluxogramas separados para procedimentos específicos como sejam os associados com o teste da palavra lida ser nova.
- d) Complete o programa fornecido nos ficheiros indicados em que as palavras lidas dum ficheiro são armazenadas numa lista ligada, cuja interface se define em `list.h`.

No final do programa escreve-se para ficheiro a listagem das palavras, o número de ocorrências e a frequência de ocorrências correspondentes:

(<frequência>=<número de ocorrências da palavra> / <número total de palavras>)

A escrita é feita percorrendo a lista ligada do início para o fim.

Assegure também a correta libertação da memória alocada. Corra o Valgrind para verificar que a gestão de memória dinâmica está bem efetuada.

Compile e execute o programa com os exemplos fornecidos ou outros e confirme a sua correção.

- e) A implementação do módulo de palavras disponibilizada (`words.c`) baseia-se como vimos numa estruturação sob a forma de uma lista ligada para guardar a informação das palavras já vistas e o seu número de ocorrências. Se quisesse manter a abstração mas basear a estrutura da informação numa implementação em tabelas como foi feito em 2.1, indique que alterações e de que forma teriam de ser feitas para obter um `words_tables.c` (se o interface se mantivesse, o mesmo `words.h` poderia ser usado).

## 2.3 Discussão Comparativa

Nas seções 2.1 e 2.2 implementámos duas soluções para o problema UNIQPAL introduzido neste laboratório. Como compara as implementações apresentadas do ponto de vista de utilização de memória e complexidade computacional.